Stacks Lab

In lecture we defined the stack ADT and built two implementations, `ArrayStack` and `LinkedStack`. In this lab you use them in two programs: a checker for HTML files, and a text editor.

1. Checking that the tags of an HTML file are balanced.
2. A text editor stored in two stacks.
3. Undo and redo, with two more stacks.

The files

You write the files in `tags/` and `editor/`. The top folder holds the programs you run.

`stacks/`	`MyStack`, `ArrayStack`, `LinkedStack`: the stacks from lecture, plus `clear` and an iterator. Do not modify
`tags/Tags.java`	Part 1
`editor/StackEditor.java`	Part 2
`editor/Editor.java`	what every editor does; Part 3
`editor/GapEditor.java`	optional
`CheckTags.java`	checks HTML files with your Part 1
`EditorApp.java`, `Driver.java`	try your editor, in a window or with typed commands
`Test.java`	the tests
`data/`	HTML files for Part 1

Compile and run everything from the top folder. Everything compiles before you write anything.

```
javac Test.java
java Test
```

The tests are grouped by part, in the order you should write the code. A group that throws an exception, or runs for more than 5 seconds, fails, and `Test` moves on to the next group.

Part 1: matching tags

Your TA will guide you through this part.

An HTML file is text with **tags**: `<p>` opens a paragraph, `</p>` closes it. Tags nest like brackets: the last tag opened must be the first one closed.

```
<p>This is <b>bold</b> and <i>italic</i>.</p>      balanced
<p>This is <b>wrong</p></b>                          </p> closes <p>, but <b> is still open
```

Void tags are never closed: `<br>` (a line break), `<img ...>` (an image), `<meta ...>`, `<hr/>`, ...

`Tags.parse` is given: it reads the tags of a text into a `List<Tag>`. For each `Tag`: `name()` (`"p"` for both `<p>` and `</p>`), `isClosing()`, `isVoid()`, `line()`.

Write `firstError(tags)`. It returns:

- the position in `tags` of the first tag where the file is found to be wrong: a closing tag when no tag is open, or a closing tag that does not match the last one opened;
- `tags.size()` if the file ends with tags still open;
- `-1` if the tags are balanced.

Void tags are skipped. Then try it on real files:

```
javac CheckTags.java
java CheckTags data/page.html data/broken1.html
```

Real HTML lets you omit some closing tags (`</p>`, `</li>`, ...). The files in `data` close every tag that is not void.

Part 2: an editor on two stacks

An editor holds a text and a cursor between two characters. `StackEditor` stores the text in two stacks, one on each side of the cursor:

```
text "abcde", cursor between c and d:     abc|de

    before:  a b c      top: c, the character just before the cursor
    after:   e d        top: d, the character just after the cursor
```

Typing pushes on `before`. Moving the cursor moves one character from one stack to the other.

In `editor/StackEditor.java`, write the methods in the order of the file. `Editor.java` describes what each one does.

- `insert`, `delete`, `left`, `right`, `cursor`: `O(1)`.
- `left` at the start of the text, and `right` at the end, do nothing.
- `textBeforeCursor` is `before` from bottom to top, `textAfterCursor` is `after` from top to bottom. Read them with a for-each loop (see below).

Reading a stack. In this lab `MyStack` is `Iterable`: a for-each loop visits the elements from the top to the bottom, without changing the stack. The ADT restricts how a stack **changes**, only at the top; reading it does no harm, and in practice stacks usually allow it.

Another way to write an editor: the text in an `ArrayList<Character>`, the position of the cursor in an `int`, and typing is `add(cursor, c)`. What does typing one character at the start of a text of `N` characters cost? And typing `N` characters there?

`LinkedStack` or `ArrayStack`? `StackEditor` uses `LinkedStack`; switching is changing the two `new LinkedStack<>()`. `ArrayStack` is slightly faster and uses less memory, but when its array is full one `push` copies it: `O(1)` amortized, `O(N)` for that keystroke. `LinkedStack` takes the same time for every `push`: its performance is predictable.

Try it

```
javac EditorApp.java
java EditorApp
```

Type, Enter, Backspace, left and right arrows; Ctrl+Z undo, Ctrl+Y redo (Cmd on a Mac) after Part 3; Esc quits.

If the window does not open on your machine, `Driver` does the same with typed commands:

```
javac Driver.java
java Driver
```

Type `help` for the commands: `type some text`, `left 3`, `back`, `undo 2`, ... After each command it prints the text with `|` at the cursor and newlines as `\n`, as the tests do: `ab|c\nd`. It is also useful to reproduce a failing test step by step.

Part 3: undo and redo

Editors implement undo with a stack: every edit is pushed on it, and undo pops the last one and applies its inverse. `Editor` implements undo and redo once for every kind of editor, using only the methods of Part 2.

An edit is an `Edit` (given, in `Editor.java`): the character `c`, its position `pos` in the text, and whether it was `inserted` or deleted. Inserting `c` at `pos` and deleting the character at `pos` are inverses of each other.

In `editor/Editor.java`:

- Two stacks of `Edit`, as fields: one for undo, one for redo.
- `type` and `backspace` also record their edit on the undo stack. A new edit empties the redo stack (`clear`): after an edit, there is nothing to redo.
- `undo` pops an edit from the undo stack, applies its inverse, and pushes it on the redo stack.
- `redo` does the opposite: pops from the redo stack, applies the edit again, pushes on the undo stack.
- With nothing to undo or redo, they do nothing.

The cursor may have moved since the edit: undo first moves the cursor to the position of the edit, as real editors do. Write a private method that moves the cursor to a given position with `left` and `right`. Undo and redo change the text with `insert` and `delete`, not `type` and `backspace`: why?

Optional: a gap buffer

Some editors, Emacs for example, store the text in one array with a block of free cells, the **gap**, at the cursor:

```
text "abcdef", cursor between c and d, capacity 10

index   0 1 2 3 4 5 6 7 8 9
buf    [a b c . . . . d e f]
              ^       ^
         gapStart    gapEnd
```

Typing writes in `buf[gapStart]` and moves `gapStart` right. Moving the cursor moves one character across the gap. When the gap is full, the array doubles, as in `ArrayStack`, and the characters after the gap move to the end of the new array. It is your two stacks in a single array: `before` grows from the left, `after` from the right, toward each other.

Write `GapEditor`, then `java Test gap` and `java EditorApp gap`.

What to submit

Upload the whole lab folder to Brightspace, with `tags/Tags.java`, `editor/StackEditor.java` and `editor/Editor.java` completed.